ADMS @ VLDB 2026

HetMatch: GPU-Accelerated Heterogeneous String Pattern Matching

Abigale Kim
University of Wisconsin–Madison

Xiangyao Yu
University of Wisconsin–Madison

GPUs: the Future of Data Analytics



GPU String Processing

- Significant speedups for string query processing on GPUs compared to CPUs
  - GPU's hyper parallelism contributes to fast query processing
- GPUs are not optimal when threads do uneven amounts of work
- Strings are variable length and prone to skew
- Pattern matching is a representative problem in string processing

Research Question: How should we parallelize pattern matching on the GPU?

Pattern Matching

- Determines whether a substring (pattern) occurs within each string of text data

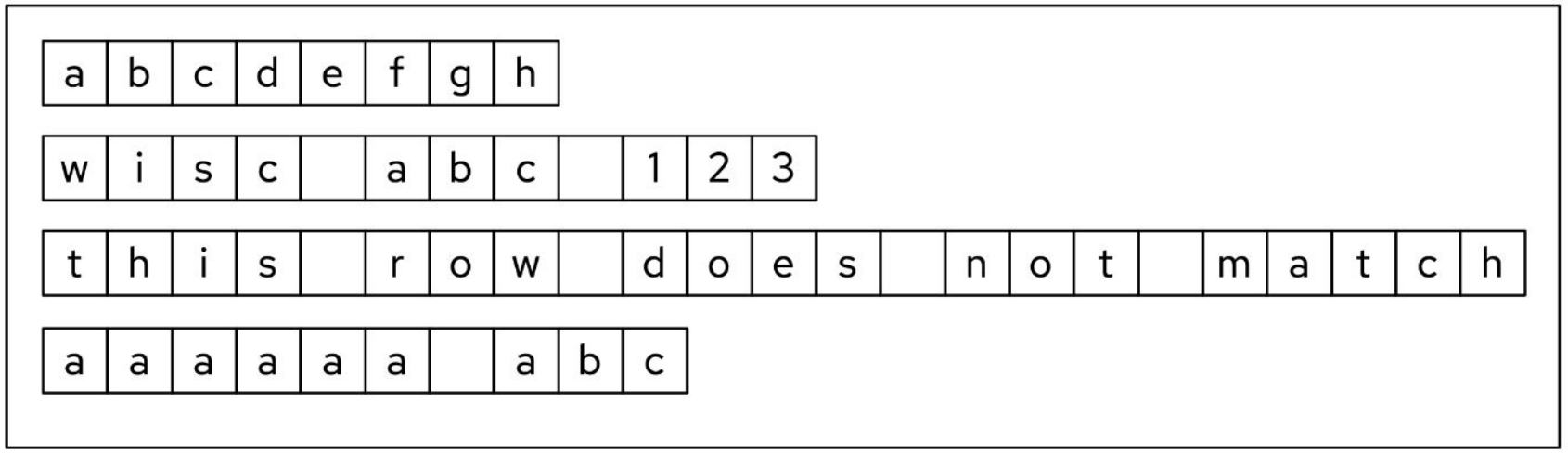
```
import cudf # NVIDIA's dataframe library

df = cudf.DataFrame({
    "message": [
        "abcdefgh",
        "wisc abc abc",
        "this row does not match",
        "aaaaaa abc",
    ]
})

mask = df["message"].str.contains("abc")
# mask = [True, True, False, True]
```

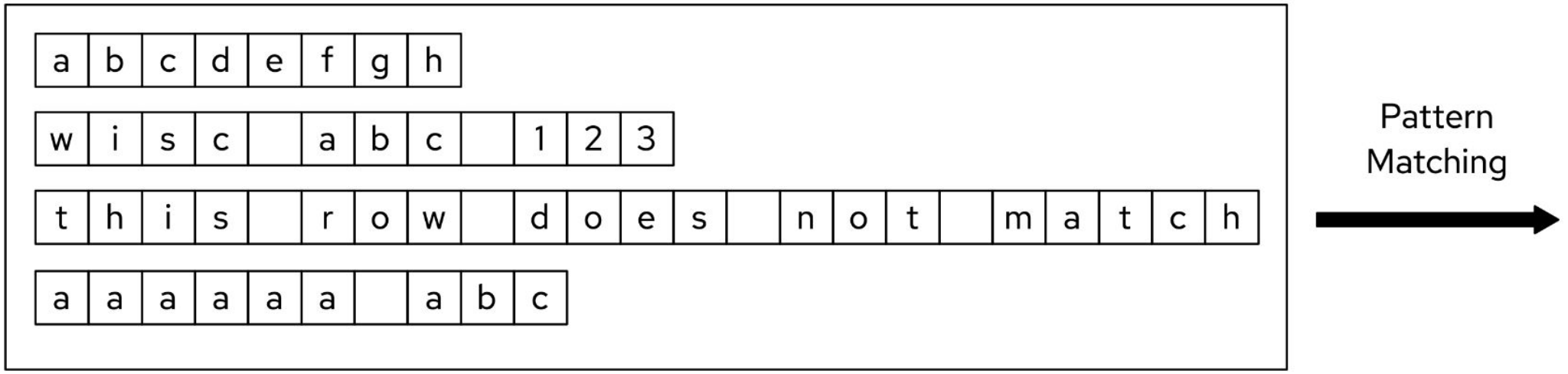
Pattern Matching

Input Data pattern = 'abc'



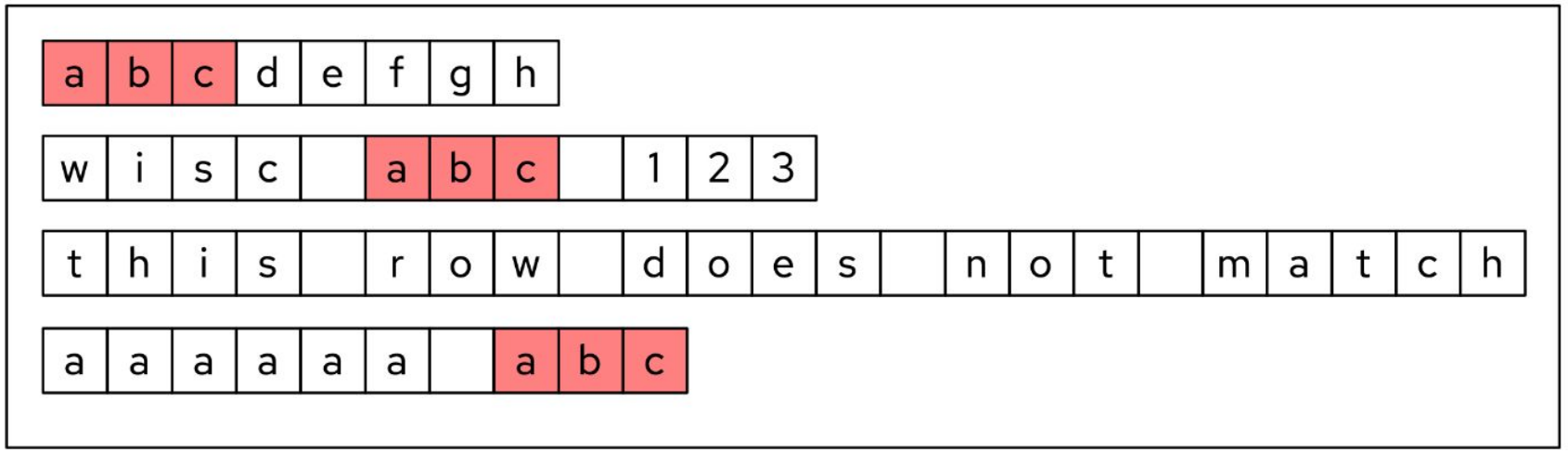
Pattern Matching

Input Data pattern = 'abc'



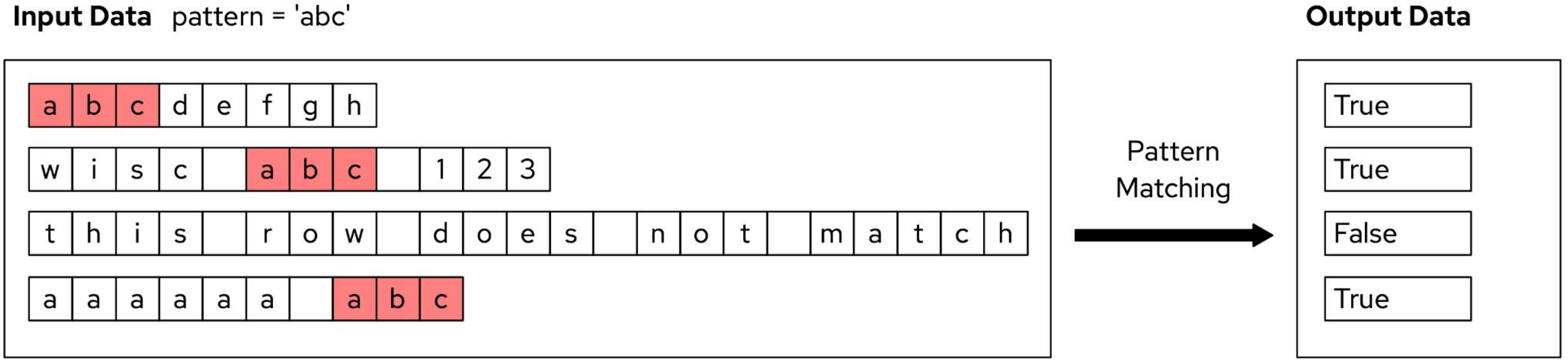
Pattern Matching

Input Data pattern = 'abc'



Pattern Matching

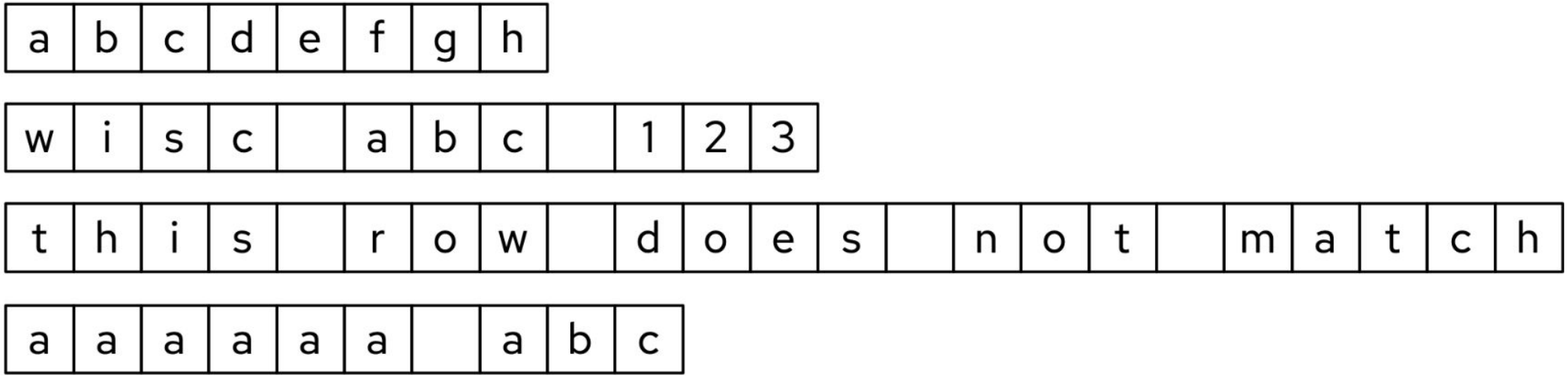
Pattern Matching



- GPU pattern matching implementations assign one thread per string or one warp per string

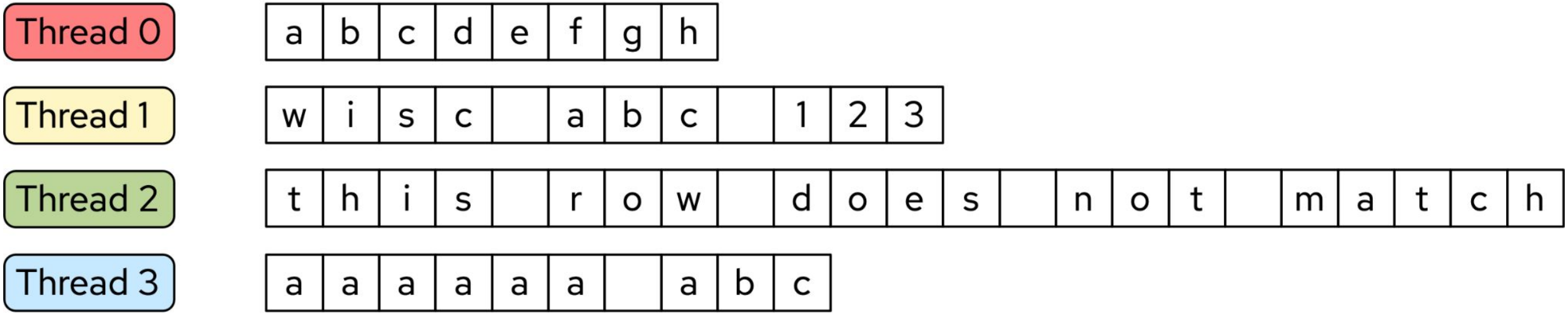
Thread per String

pattern = 'abc'

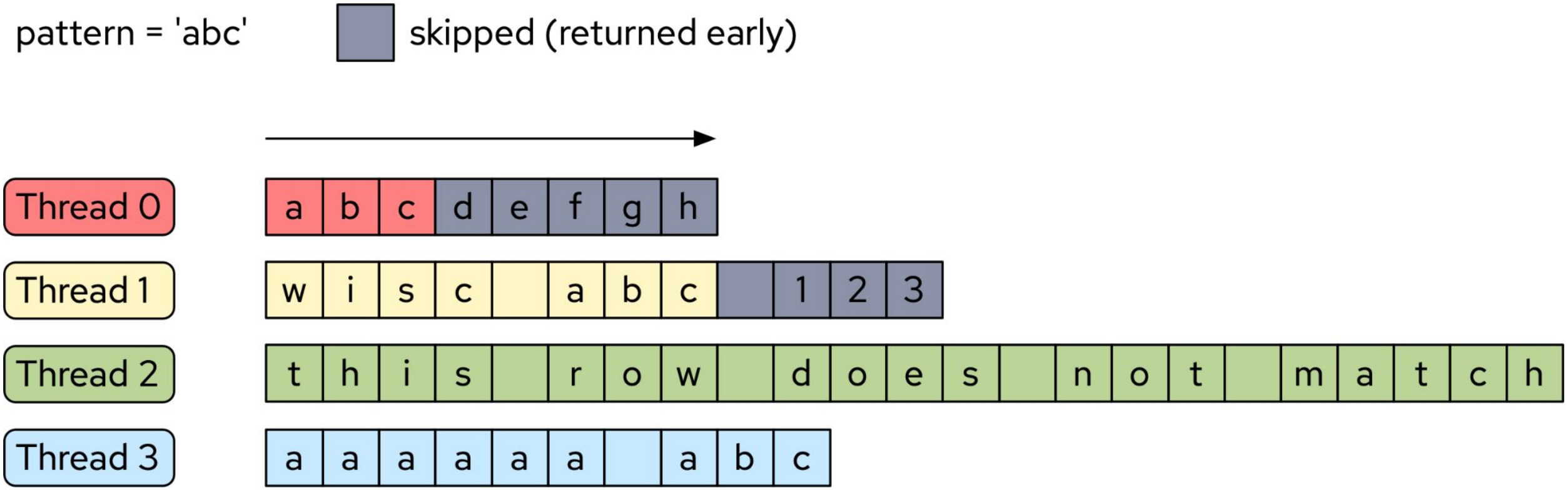


Thread per String

pattern = 'abc'



Thread per String



Thread-per-String Tradeoffs

Pros

- No cross-thread coordination needed ✅
- Efficient for short strings with uniform length ✅

Cons

- Not coalesced memory accesses ❌
- Performance degrades with non-uniform string length ❌

Warp per String

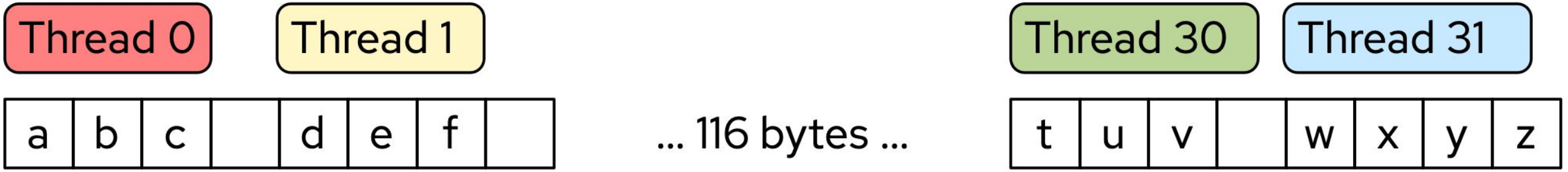
pattern = 'abc'

Warp: a group of 32 threads that the GPU schedules to execute together

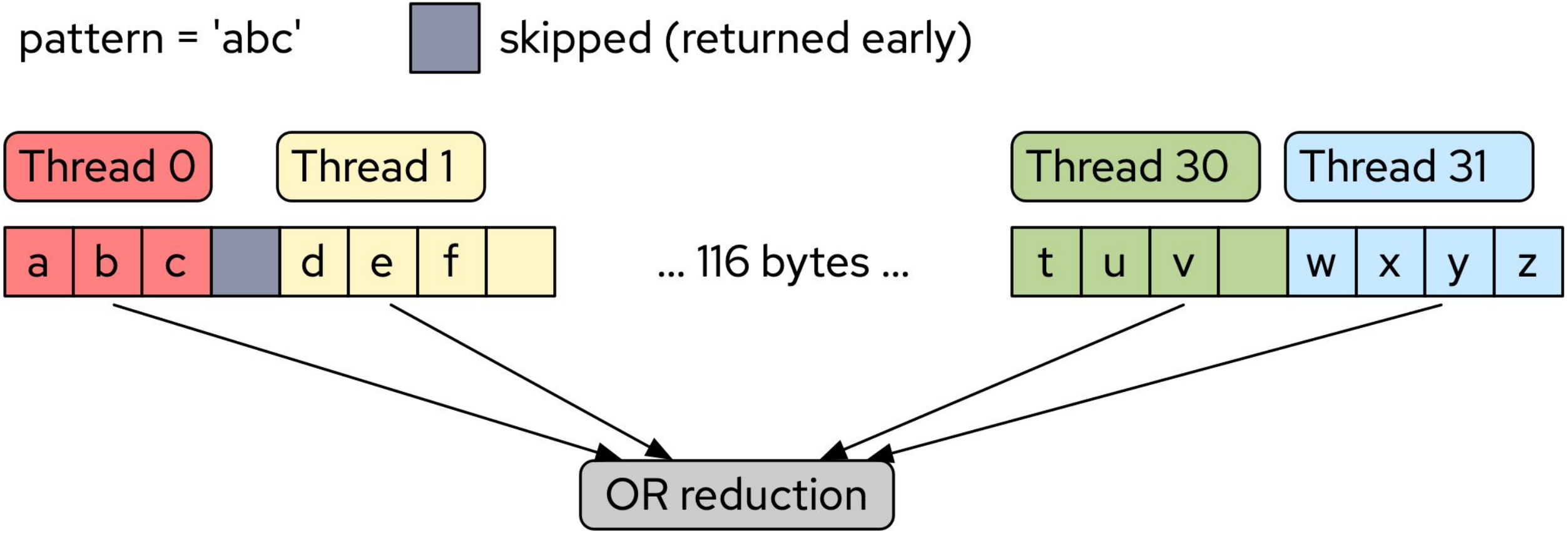


Warp per String

pattern = 'abc'



Warp per String



Warp-per-String Tradeoffs

Pros

- More efficient memory access pattern ✅
  - Warp accesses one cache line per iteration
- Efficient for long strings ✅

Cons

- Requires thread synchronization ❌
- Wasted work when string length < 128 bytes ❌

NVIDIA cuDF

- NVIDIA's GPU-accelerated DataFrame library
- Replacement for Pandas
- Used as the backend of GPU database systems (Sirius, BlazingSQL, Spark RAPIDS, Velox-cuDF) and ML frameworks (cuML)

cuDF `contains` Implementation

```
Input: string column C, pattern p
Output: Boolean array R

m = MeanLength(C)
R = BoolColumn(C.size())
if (m <= 64) {
   thread_per_string_kernel(C, p, R)
} else {
   warp_per_string_kernel(C, p, R)
}
return R
```

HetMatch Motivation

abcdefgh (length=8)

gpu gpu gpu (length=8)

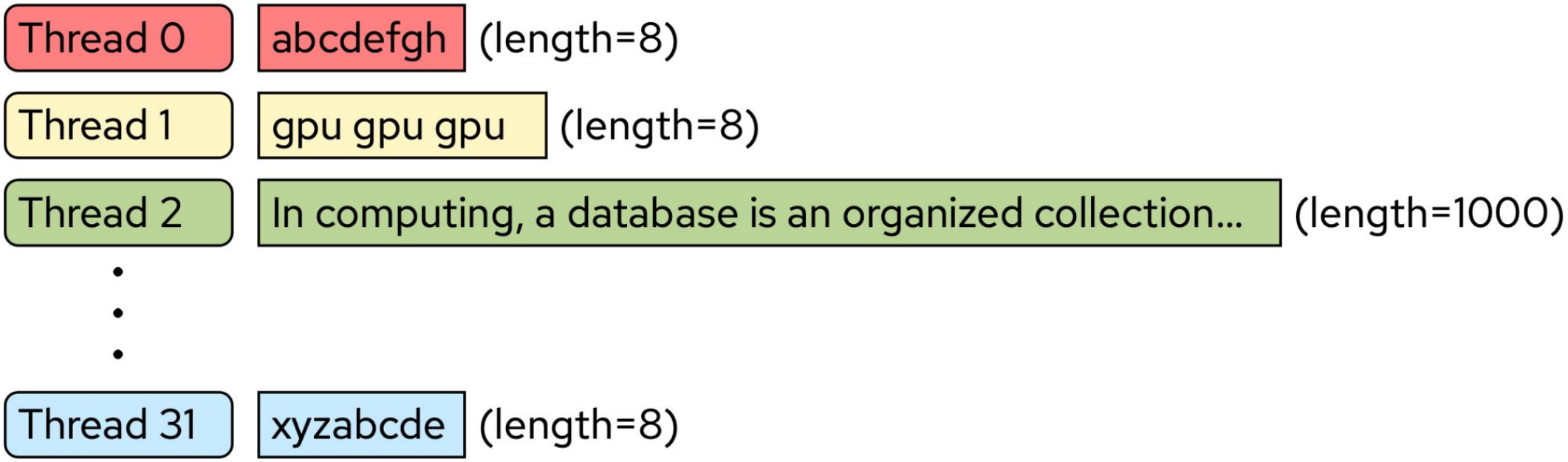
In computing, a database is an organized collection... (length=1000)

•
•
•

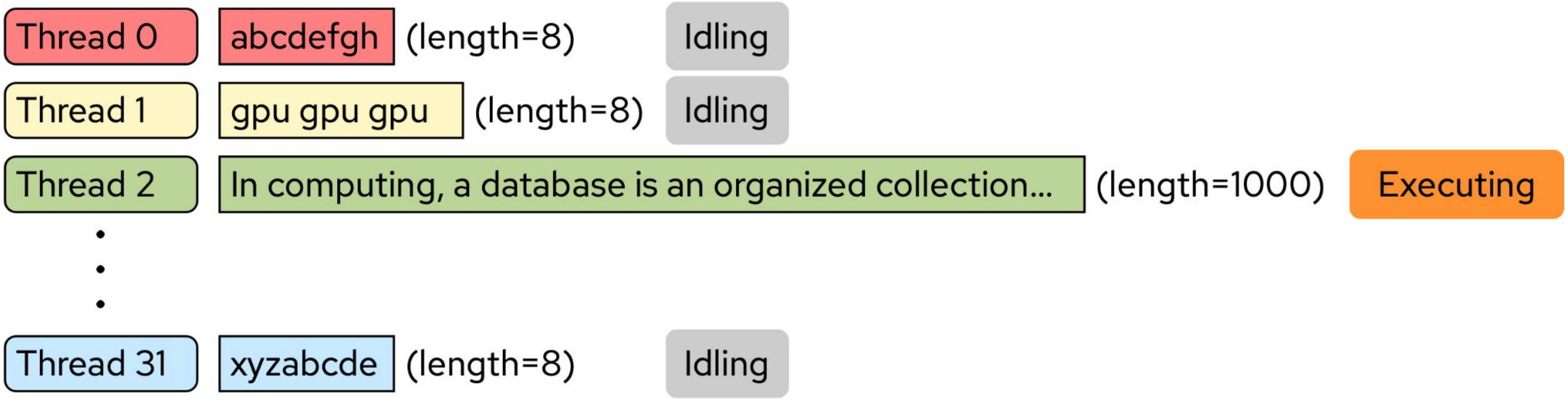
abcdefgh (length=8)

Average length = 60 bytes

HetMatch Motivation



HetMatch Motivation



HetMatch Algorithm

```
Input: string column C, pattern p
Output: Boolean array R

R = BoolColumn(C.size())
[I_short, I_long] = Partition(I, i < 128)
thread_per_string_kernel(C, p, I_short, R)
warp_per_string_kernel(C, p, I_long, R)
return R
```

HetMatch Example

Column c, p = 'abc'

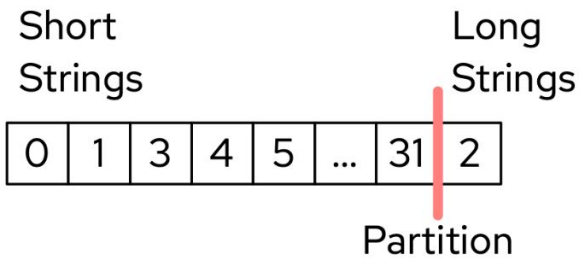
idx	value	
0	abcdefgh	(length=8)
1	gpu gpu gpu	(length=8)
2	In computing, a database...	(length=1000)
	•	
	•	
	•	
31	abcdefgh	(length=8)

HetMatch Example

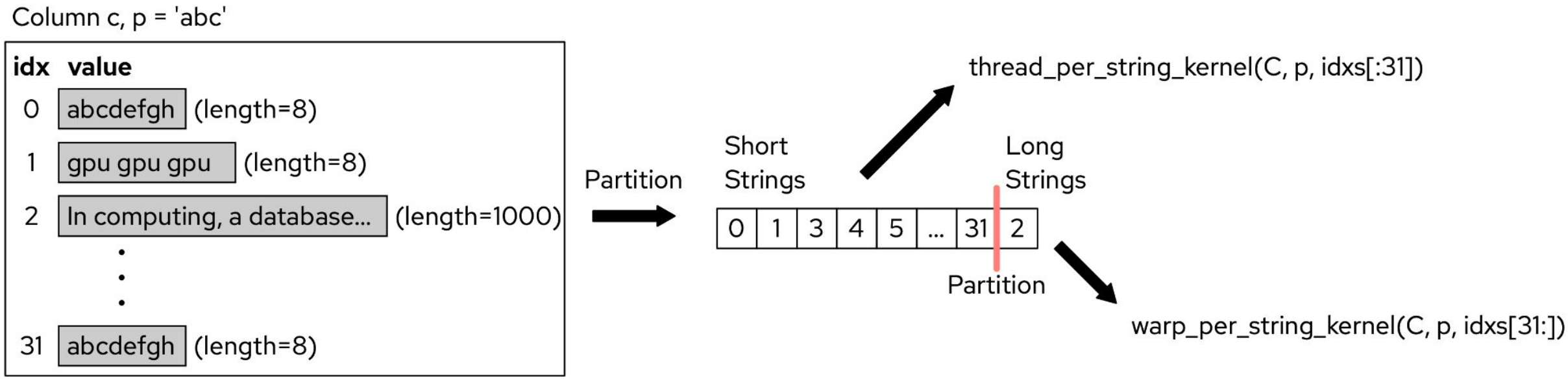
Column c, p = 'abc'

idx	value	
0	abcdefgh	(length=8)
1	gpu gpu gpu	(length=8)
2	In computing, a database...	(length=1000)
⋮		
31	abcdefgh	(length=8)

Partition



HetMatch Example



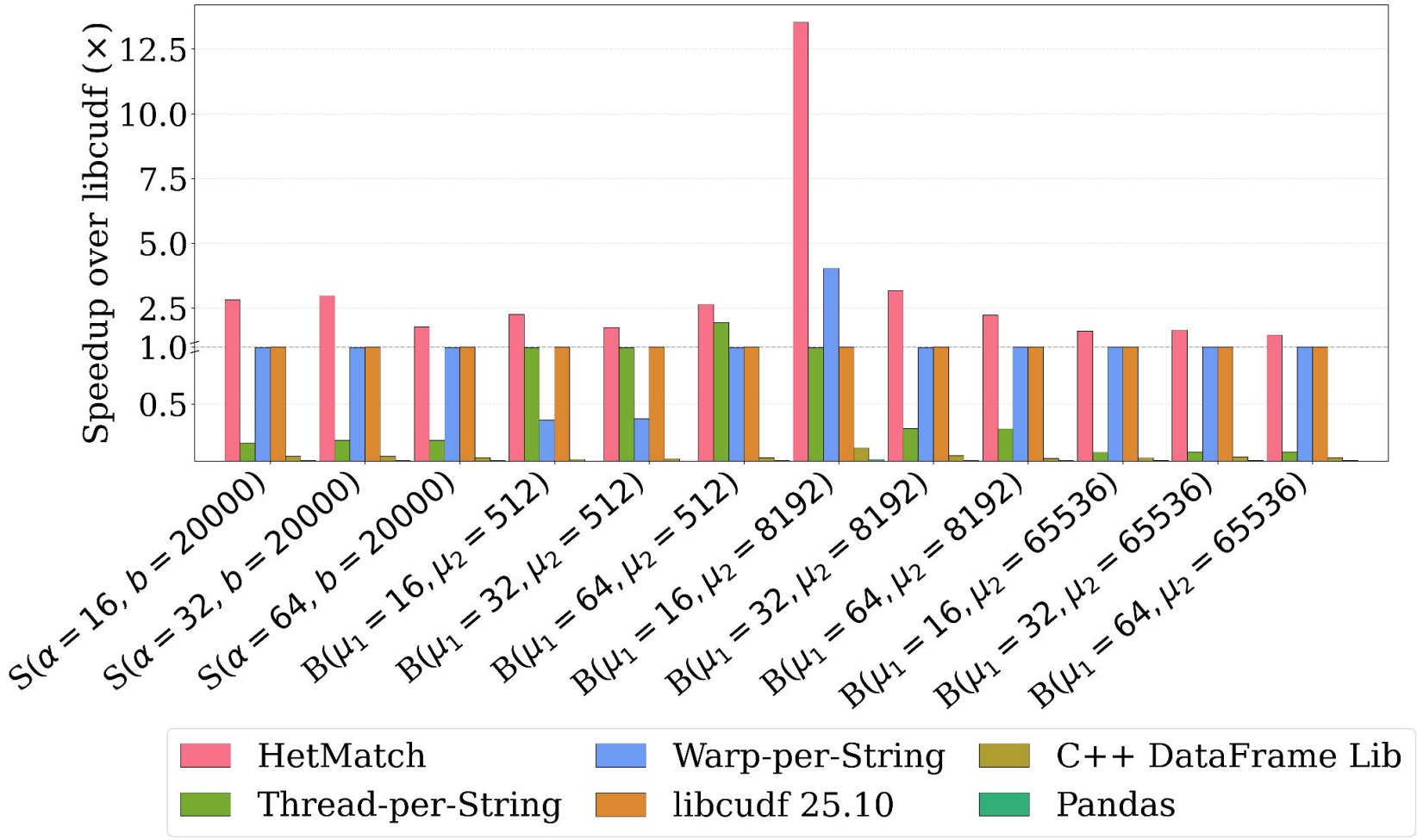
Implementation Decisions

- 128 byte threshold instead of 64 byte threshold
  - Warp per string kernel is cache line optimized, and strings with length less than 128 bytes will incur wasted work
- Unstable partition
  - Benchmarks showed algorithmic overhead of stable partition was less than the benefit of spatial locality
- Built directly into cuDF's contains operator
  - Currently an active PR in cuDF's GitHub repository!

Evaluation

- NVIDIA H100, 80 GB HBM, 2000 GB/s memory bandwidth
- Intel Xeon Gold 6530 CPU, 32 cores, 240 GB DRAM
- Ubuntu 22.04, CUDA 13.0
- Synthetic datasets:
  - Skewed (a, b): 99.5% of strings with length a (16, 32, 64) bytes, the rest with length b (20000) bytes
  - Bimodal ($\mu1$, $\sigma1$, $\mu2$, $\sigma2$): 99.5% of strings drawn from N(μ_1, σ_1) the rest from N(μ_2, σ_2) , where $\mu2 > \mu1$
    - Smaller distributions: N(16, 5), N(32,10), N(64,20)
    - Larger distributions: N(512,100), N(8192,500), N(65536,2000)

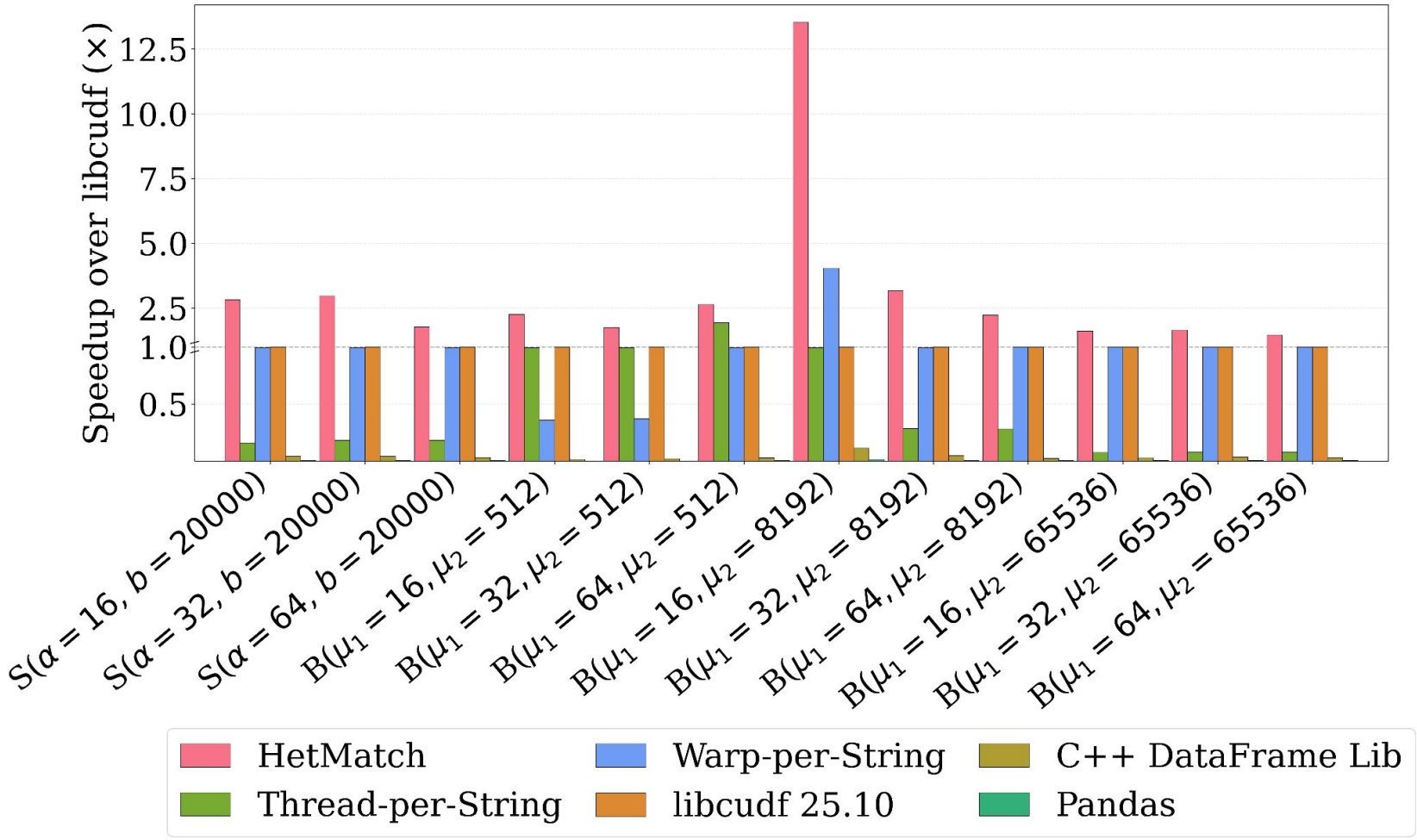
Evaluation: Synthetic Workloads



Avg. speedup: 2.5x

Max speedup: 13.5x

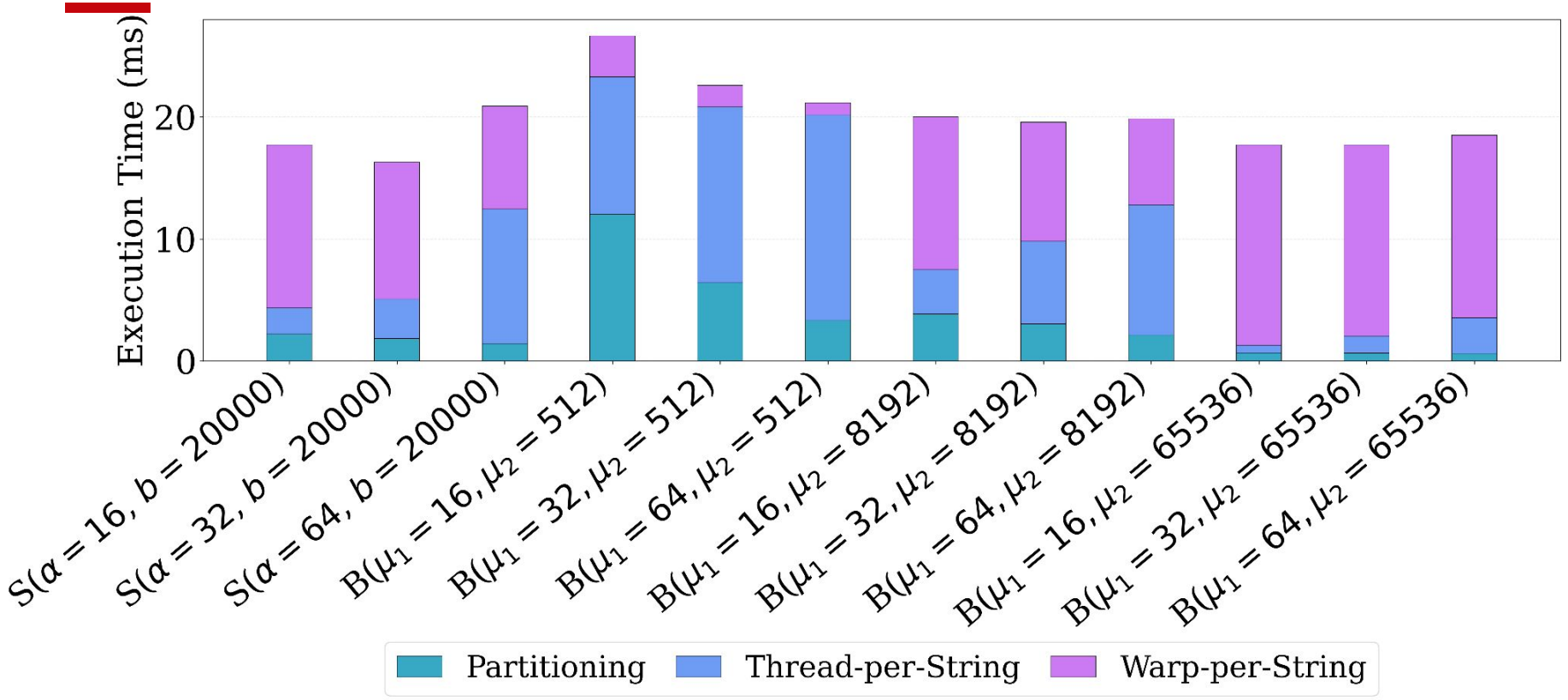
Evaluation: Synthetic Workloads



Takeaway:

HetMatch is consistently faster than cuDF on highly skewed distributions.

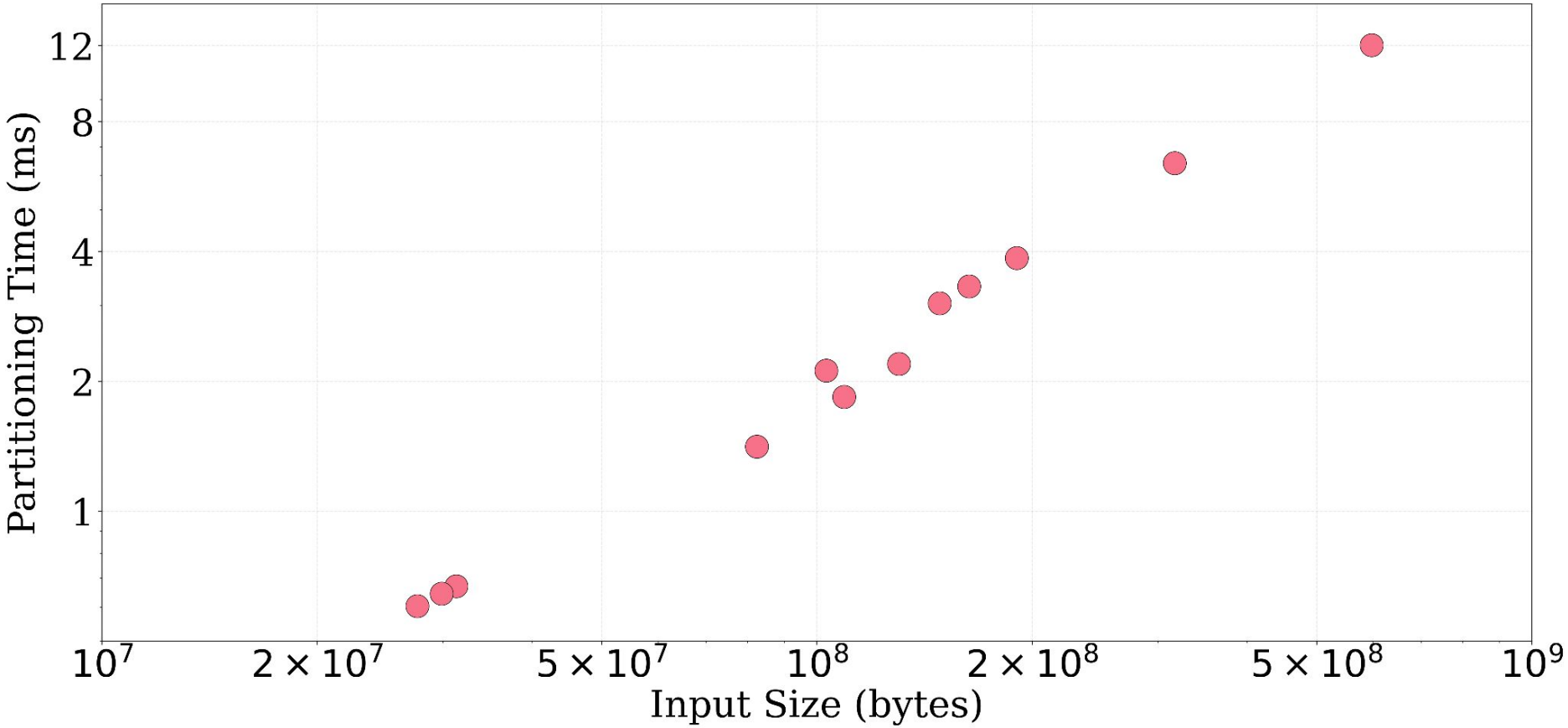
Evaluation: Partitioning Breakdown



Avg. pct. of partitioning overhead: 14.7%

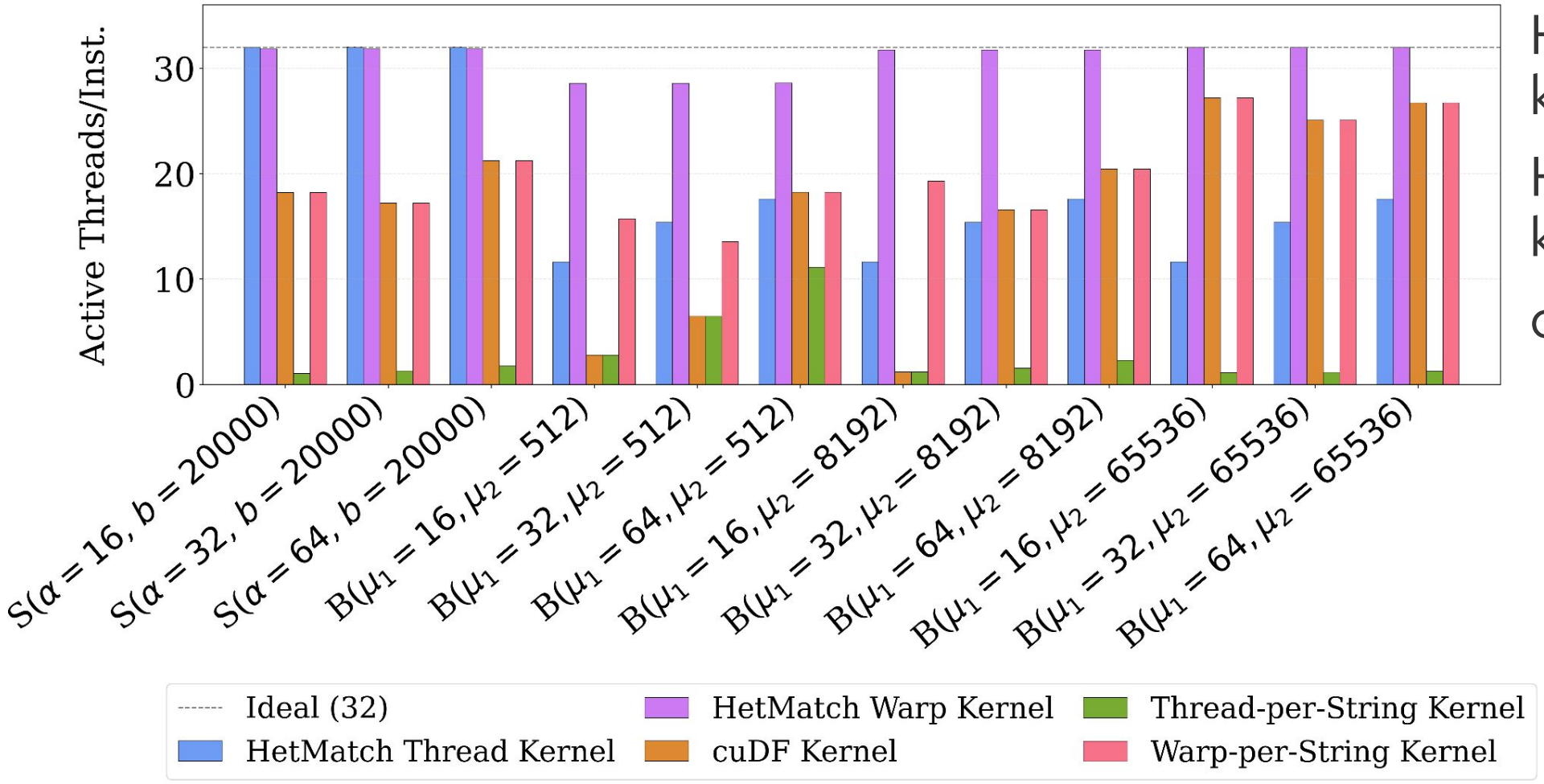
Optimization: combine thread-per-string kernel & partitioning phase

Evaluation: Partitioning Breakdown



Takeaway: Partitioning cost scales linearly, and is worth the performance gain during the pattern matching phase.

Evaluation: Active Threads per Instruction

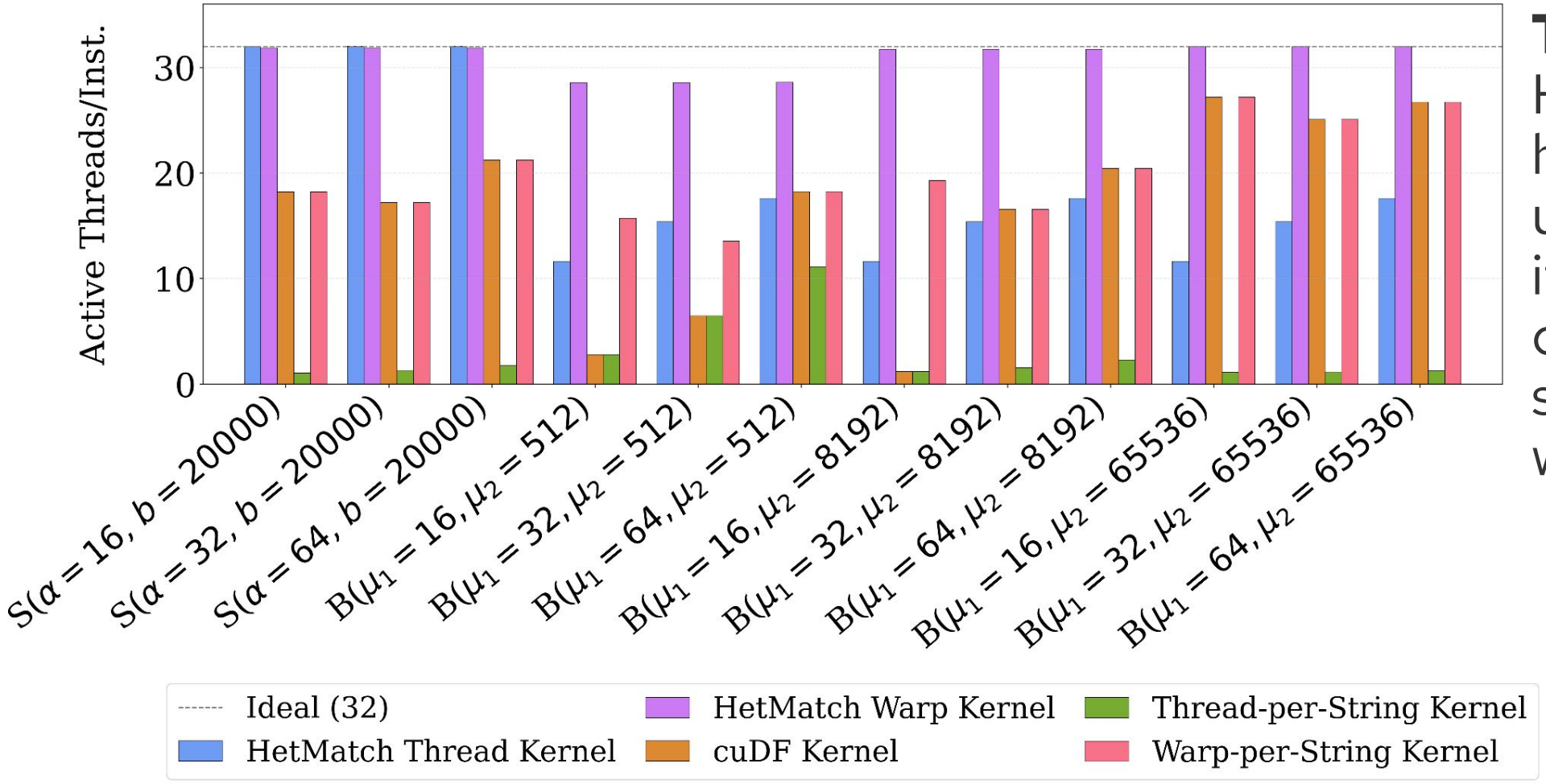


HetMatch thread kernel avg: 20.2

HetMatch warp kernel avg: 29.6

cuDF avg: 16.8

Evaluation: Active Threads per Instruction



Takeaway: HetMatch's higher thread utilization makes it faster than cuDF on highly skewed workloads.

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with many short strings:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
Twitter Posts	88.9%	11.1%	74	69	484	982
Common URLs	92.1%	7.9%	77	65	147	256
Shopify Product Names	91.0%	9.0%	60	47	168	198

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with many short strings:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
Twitter Posts	88.9%	11.1%	74	69	484	982
Common URLs	92.1%	7.9%	77	65	147	256
Shopify Product Names	91.0%	9.0%	60	47	168	198

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with many short strings:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
Twitter Posts	88.9%	11.1%	74	69	484	982
Common URLs	92.1%	7.9%	77	65	147	256
Shopify Product Names	91.0%	9.0%	60	47	168	198

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with many short strings:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
Twitter Posts	88.9%	11.1%	74	69	484	982
Common URLs	92.1%	7.9%	77	65	147	256
Shopify Product Names	91.0%	9.0%	60	47	168	198

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with very long tails:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
GitHub Commits	30.9%	69.1%	430	280	1300	2563
LLM Chat Logs	38.9%	61.1%	711	263	3177	5132

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with very long tails:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
GitHub Commits	30.9%	69.1%	430	280	1300	2563
LLM Chat Logs	38.9%	61.1%	711	263	3177	5132

Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with very long tails:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
GitHub Commits	30.9%	69.1%	430	280	1300	2563
LLM Chat Logs	38.9%	61.1%	711	263	3177	5132

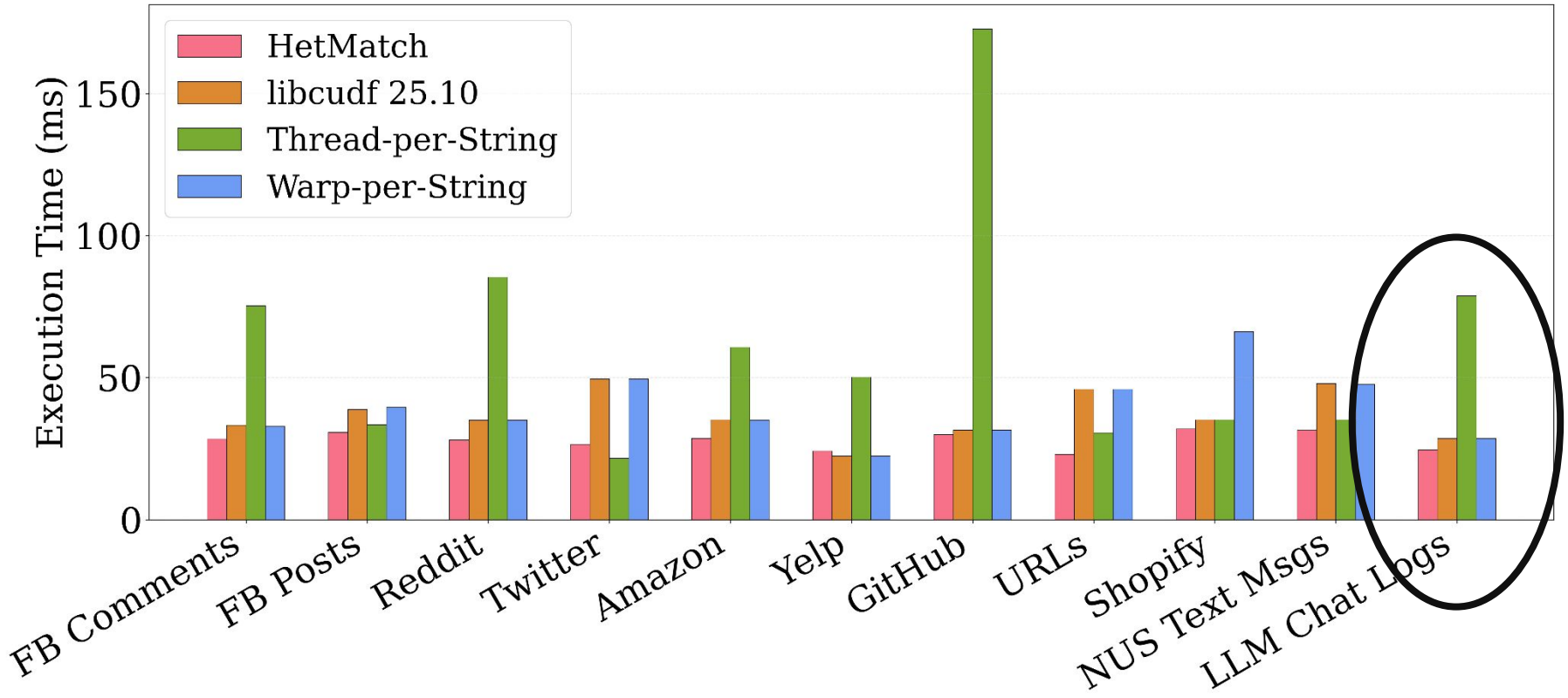
Evaluation: Real World Workloads

- Selected string datasets with natural length skew
- Most datasets are right-skewed (many short strings, few longer strings)

Datasets with very long tails:

Dataset	≤ 128 bytes	> 128 bytes	Mean	Median	95th Pct.	99th Pct.
GitHub Commits	30.9%	69.1%	430	280	1300	2563
LLM Chat Logs	38.9%	61.1%	711	263	3177	5132

Evaluation: Real World Workloads

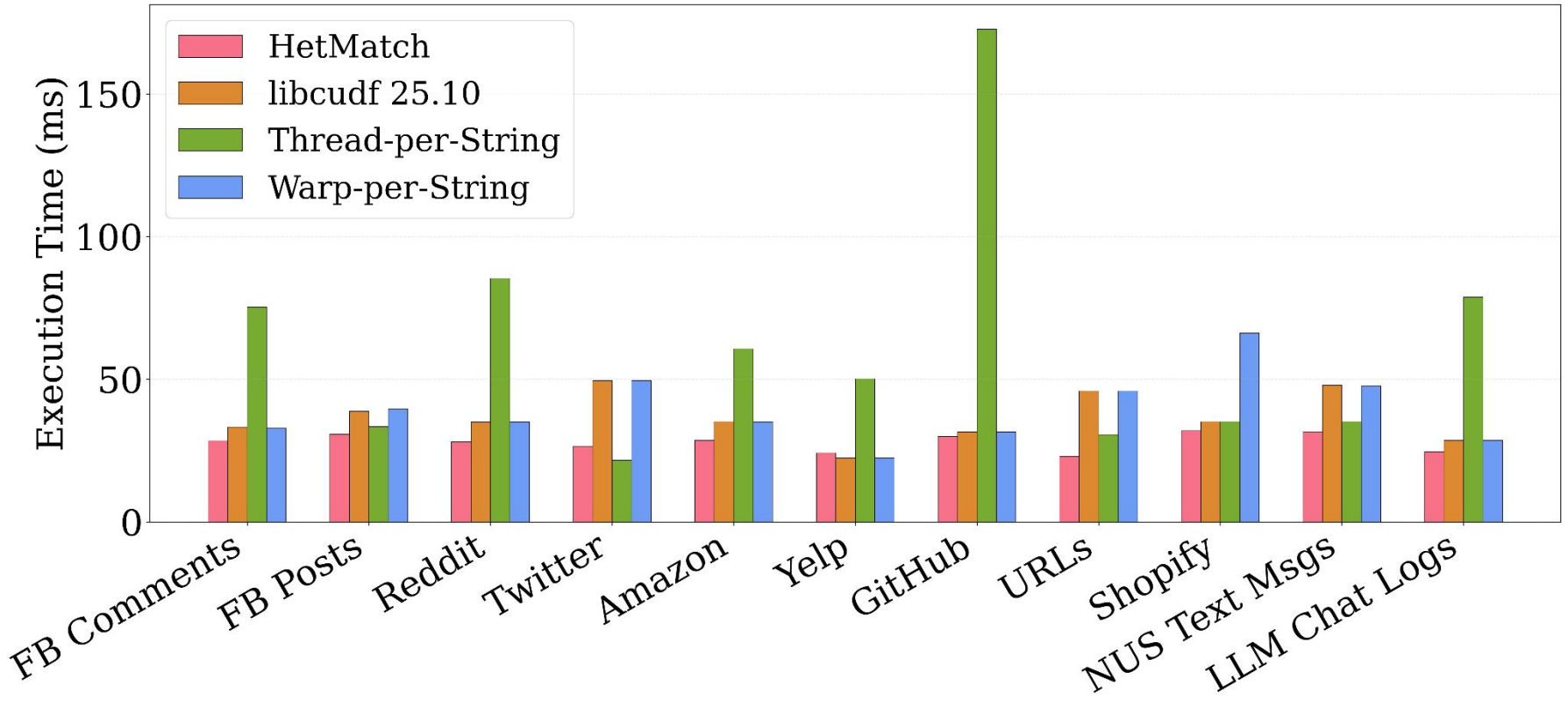


Avg speedup: 1.3x

Max speedup: 2x

LLM chat logs speedup: 1.2x

Evaluation: Real World Workloads



Takeaway: On real-world datasets with natural skew, HetMatch's approach still provides significant speedup.

Conclusion

- cuDF's existing approach is suboptimal for highly skewed workloads
- We present HetMatch, a new heterogeneous pattern matching approach built directly into NVIDIA's cuDF
- Our approach gets up to 13.5x speedup on synthetic workloads and 2x speedup on real-world workloads

- cuDF PR: github.com/NVIDIA/cudf/pull/21917
- Benchmarking code: github.com/abigalekim/contains_bench
- Contact: abigale@cs.wisc.edu